Multiple User Login Scenario

Scenario:

Test login functionality using multiple usernames and passwords.

Task:

Use @DataProvider.

Test Data:

admin / admin123

user / user123

test / test123

Code:

```
package sdet_selenium_day_25;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

public class SauceDemoLoginTest {

   WebDriver driver;

   @BeforeMethod
   public void setUp() {
      // Initializes ChromeDriver (Ensure WebDriverManager or path is set)
```

```
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.get("https://saucedemo.com");
    }

    @DataProvider(name = "loginData")
    public Object[][] getLoginData() {
        return new Object[][] {
            { "admin", "admin123" },
            { "user", "user123" },
            { "test", "test123" }
        };
    }

    @Test(dataProvider = "loginData")
    public void testMultipleLogins(String username, String password) {
        // Locate elements and input data
        driver.findElement(By.id("user-name")).sendKeys(username);
        driver.findElement(By.id("password")).sendKeys(password);
        driver.findElement(By.id("login-button")).click();

        // SauceDemo throws an error message for these invalid credentials
        boolean isErrorDisplayed = driver.findElement(By.cssSelector("[data-
test='error']")).isDisplayed();

        // Assert that the error message appears for bad credentials
        Assert.assertTrue(isErrorDisplayed, "Error message was not displayed for credentials: " +
username);
    }
```

```
    @AfterMethod
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

2. Smoke and Regression Scenario

Scenario:

Company wants to execute smoke tests separately.

Task:

Create:

- 2 smoke test methods
- 2 regression test methods

Use groups.

Code:

```
package sdet_selenium_day_25;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
```

```
public class ExecutionGroupsTest {

   WebDriver driver;

   @BeforeMethod(alwaysRun = true)
   public void setUp() {
      driver = new ChromeDriver();
      driver.manage().window().maximize();
      driver.get("https://saucedemo.com");
   }

   // --- SMOKE TESTS ---

   @Test(groups = { "smoke" })
   public void testLoginPageLoading() {
      boolean isLogoDisplayed = driver.findElement(By.className("login_logo")).isDisplayed();
      Assert.assertTrue(isLogoDisplayed, "Login page failed to load corporate branding.");
   }

   @Test(groups = { "smoke" })
   public void testValidStandardLogin() {
      driver.findElement(By.id("user-name")).sendKeys("standard_user");
      driver.findElement(By.id("password")).sendKeys("secret_sauce");
      driver.findElement(By.id("login-button")).click();

      String currentUrl = driver.getCurrentUrl();
      Assert.assertTrue(currentUrl.contains("inventory.html"), "Valid login failed to redirect to
dashboard.");
   }
```

```
    // --- REGRESSION TESTS ---

    @Test(groups = { "regression" })
    public void testLockedOutUserErrorMessage() {
        driver.findElement(By.id("user-name")).sendKeys("locked_out_user");
        driver.findElement(By.id("password")).sendKeys("secret_sauce");
        driver.findElement(By.id("login-button")).click();

        String errorText = driver.findElement(By.cssSelector("[data-test='error']")).getText();
        Assert.assertTrue(errorText.contains("Sorry, this user has been locked out"), "Wrong error message for locked user.");
    }

    @Test(groups = { "regression" })
    public void testPasswordVisibilityMasking() {
        String inputType = driver.findElement(By.id("password")).getAttribute("type");
        Assert.assertEquals(inputType, "password", "Password field value is exposed as plain text.");
    }

    @AfterMethod(alwaysRun = true)
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```